ISIS Neutron and Muon Source

Mantid in a Browser: a greener alternative to cloud servers?

Duc Le

ISIS

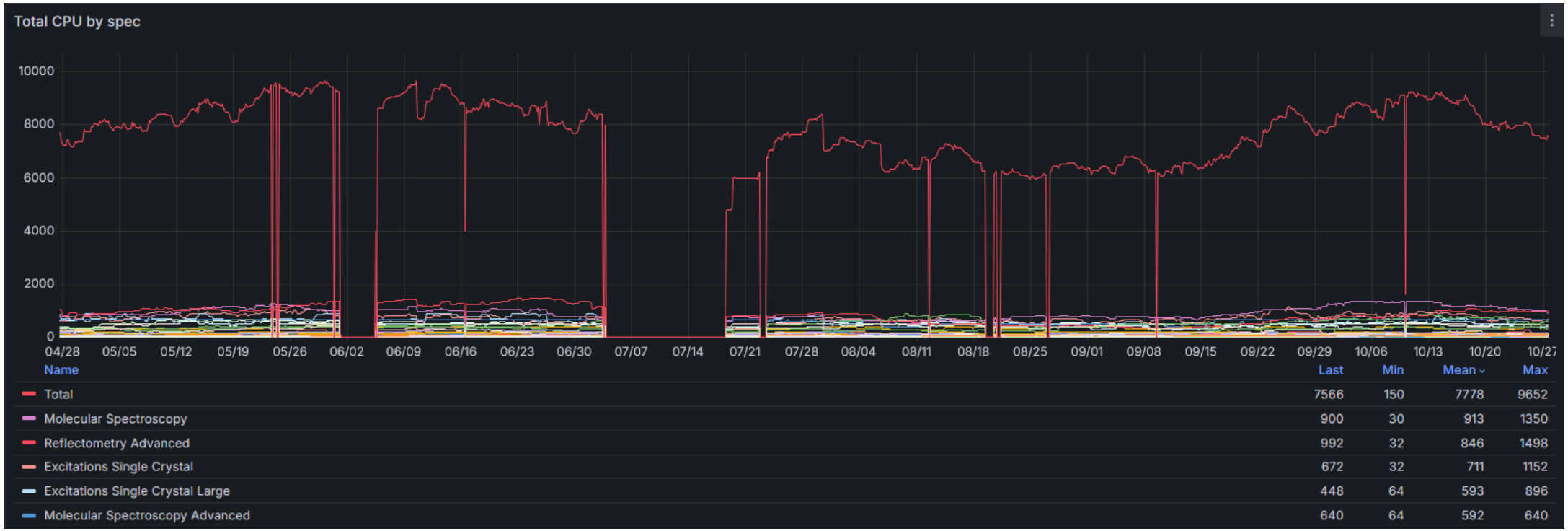
04/11/2025

(Slido)

Try it!

Introduction and Motivation

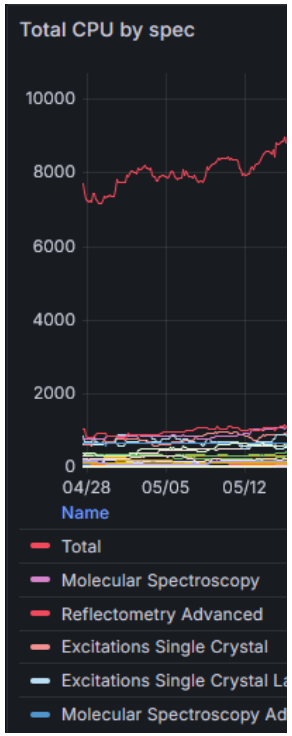
- STFC provides a cloud-based data analysis cluster accessible in a webpage via noVNC.
- ISIS used on average ~8000 cores (of ~10000) in last cycle.

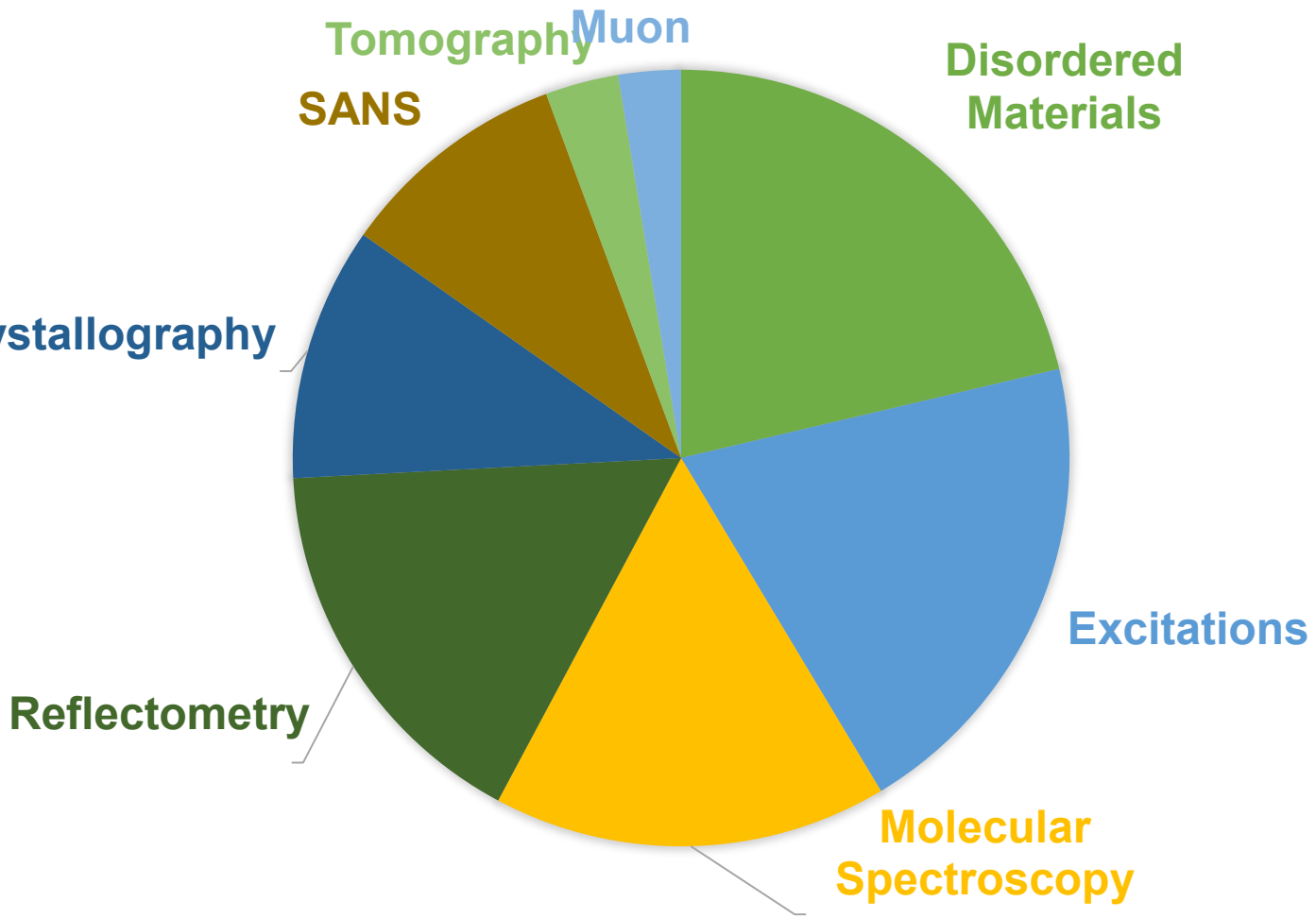


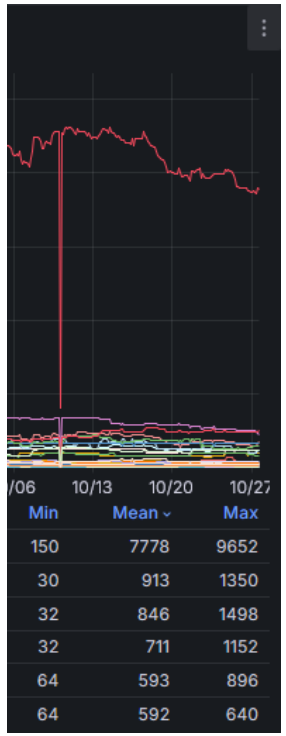
Introduction and Motivation

- STFC pro
- ISIS used

...C.







Introduction and Motivation

- STFC provides a cloud-based data analysis cluster accessible in a webpage via noVNC.
- ISIS used on average ~8000 cores (of ~10000) in last cycle.
- **Average load is only 2% of CPU and 10% of RAM** – most of the time, the cluster is idle.

ISIS Neutron and Muon Source

Science and Technology Facilities Council

Introduction and Motivation

- STFC provides a cloud-based data analysis cluster accessible in a webpage via noVNC.
- ISIS used on average ~8000 cores (of ~10000) in last cycle.
- **Average load is only 2% of CPU and 10% of RAM** – most of the time, the cluster is idle.

- Generally users create a virtual machine (VM) for one task and then do not delete it, or have several VMs running (in separate browser tabs) each for single specific task.
- Many tasks do not require a lot of CPU/RAM but the VMs are specified for the most compute intensive tasks which are only needed some of the time.

- What if users can run some of the less intensive tasks (e.g. experiment planning) in a browser tab just like the cloud system but with the computation done **on their own computer**?

PYODIDE

WA

WebAssembly

- Modern browsers have **two** built-in interpreters: for Javascript *and WebAssembly*.
  - NB: Javascript is a 32-bit platform so is limited to 4GB memory.
- C/C++/Fortran/Rust etc. code can be compiled to WebAssembly to run in a browser.
- The cross-compiling toolchain is *Emscripten*, based on LLVM.

- Pyodide is a WebAssembly compiled version of CPython, allowing Python to be run in the browser.

- It has an ecosystem of packages (wheels) and provides a build environment on top of Emscripten but only for PEP518-compliant projects (needs `pyproject.toml`).
- Pyodide/wasm wheels are currently not accepted on PyPI – but this is working its way through PEPs and likely will be approved in future.
- Each specific version of Pyodide must use a specific version of Python and Emscripten.
  - We use Pyodide-0.27.* which targets Python 3.12 and Emscripten 3.1.58
  - Newer Pyodide-0.28.* targets Python 3.13 and Emscripten 4.0.9

WebAssembly

- Emscripten also provides a runtime environment which mimics a POSIX system.
  - There is a virtual file system, and implementations of `libc`, `libcxx` etc.
  - But it has only a rudimentary dynamic linking system: Each program must consist of a single “main module” (Pyodide/CPython for us - which is automatically linked to `libc`) and dynamically loadable “side-module”s.
  - On loading, all the symbols of the side-modules are connected to the main module – so if two side modules define the same symbol, a `duplicate symbol` runtime error is raised.
  - Side modules also increases the overheads such that often Emscripten apps are compiled statically.
  - Exceptions handling currently requires a JS module (slow). A new wasm exceptions engine is available but only in Pyodide 0.28.*
  - There is pthread support for parallelism, but it was too much of a pain to compile TBB so we’ve disabled thread parallelism in micromantid.

Micromantid

- Cannot use the Mantid build system as it is – need to compile it from pyproject.toml
- Use `scikit-build` instead and almost identical CMakeFiles:
  - We choose to build a single shared-object: `_micromantid.so` instead of the set of `_kernel.so`, `_api.so`, `_dataobject.so`, `_geometry.so`, and `_curvefitting.so`.
  - But wasm has a hard max limit of 10000 exported symbols which we are close to hitting.
- Support libraries (`boost`, `poco`, `gsl`, `libhdf5`) are compiled as separate share-object libraries ("side-modules") and bundled into the wheel. (TODO: TBB)
- Mantid factory class registrations uses a `#define` macro with a dummy class whose constructor has the side-effect (as the result of the "comma operator") of calling the register method of the factory.
  - This doesn't work in Emscripten (don't know why not) – we need to call register on all algorithms, dataobjects, fit functions etc. manually… ☹
- Poco::Net doesn't work in Emscripten (need to call web APIs directly… subject to a bunch of restrictions) – so have to disable archive, config, help and usage services.
- Also need small changes for Python 3.12 (just in `funcinspect.py`) and Numpy 2.

Micromantid

- Project has main mantid repo as a git submodule – changes to the base code are kept as a set of `diffs`.
- We also have to bundle instrument files and default `Mantid.properties` in the wheel (also means that changes to these are hard for users to make).
- Have a wrapper over the Python tests in `Framework/PythonInterface/test/python/mantid` which is run in `node.js` in github-actions – `micromantid` passes 125 out of 131 tests. CI build/test job takes ~1h.

Inelastic Neutron Scatering Web Analysis Environment (Inswae)

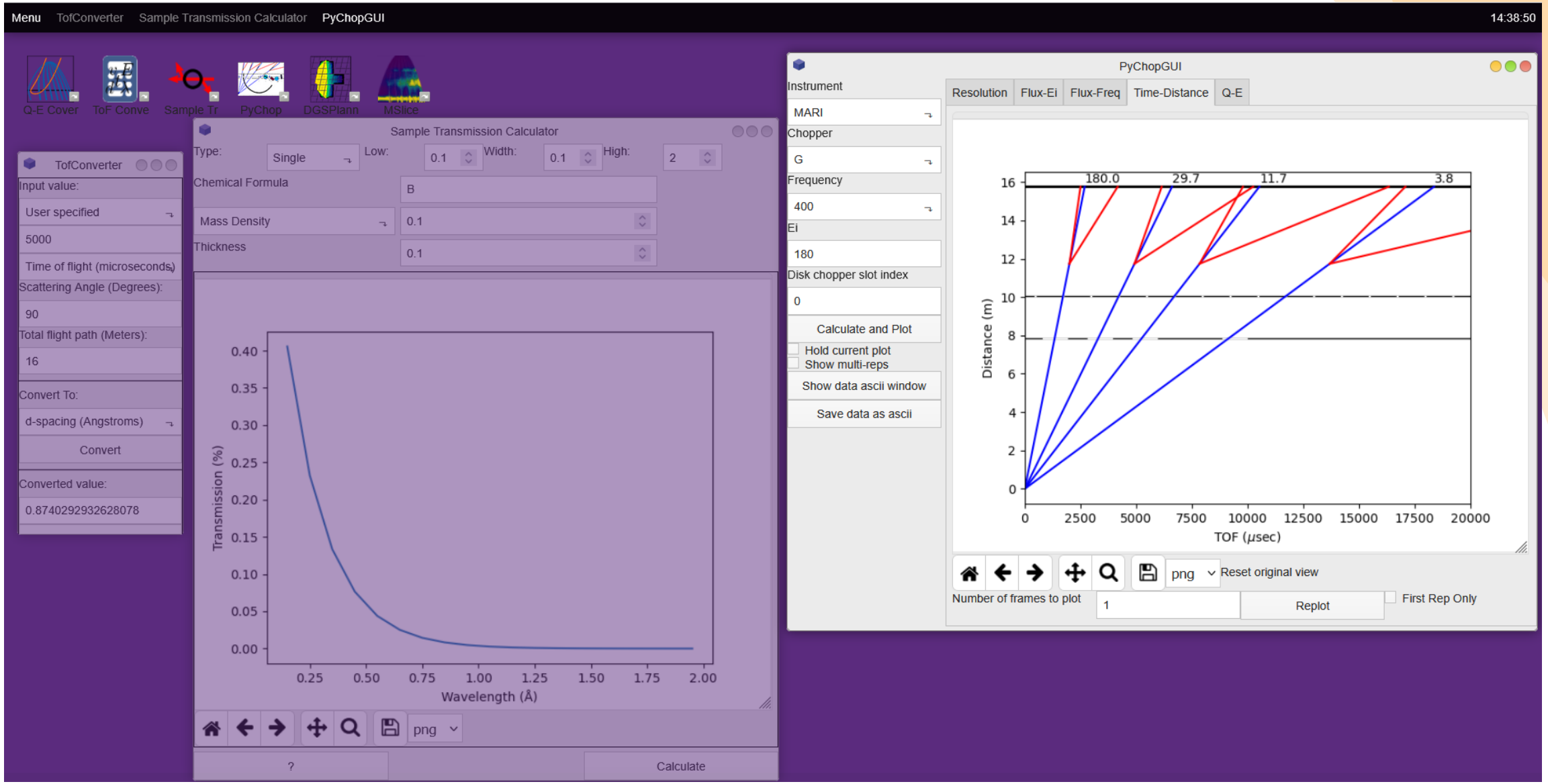
- Mantid GUI uses Qt in C++ (Workbench, several interfaces), or PyQt5.
- It is possible to compile Qt for wasm but is quite difficult, and PySide/PyQt is not fully supported.
- Mantid only uses GUI components and not the bulk of Qt's functionalities

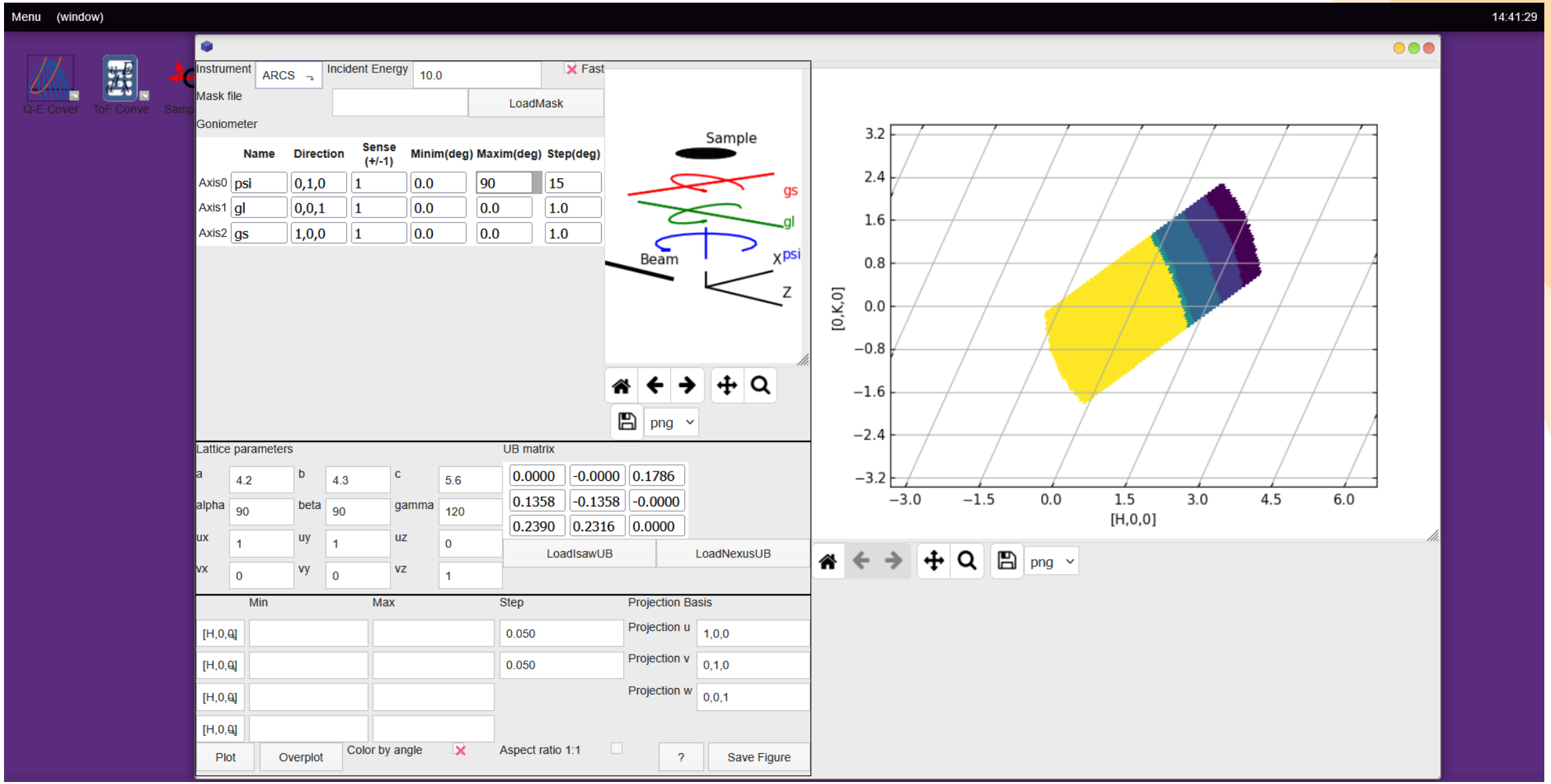
- Solution: write a Python-Javascript shim to handle the graphical components only (e.g. we don't need the webkit browser, etc.)

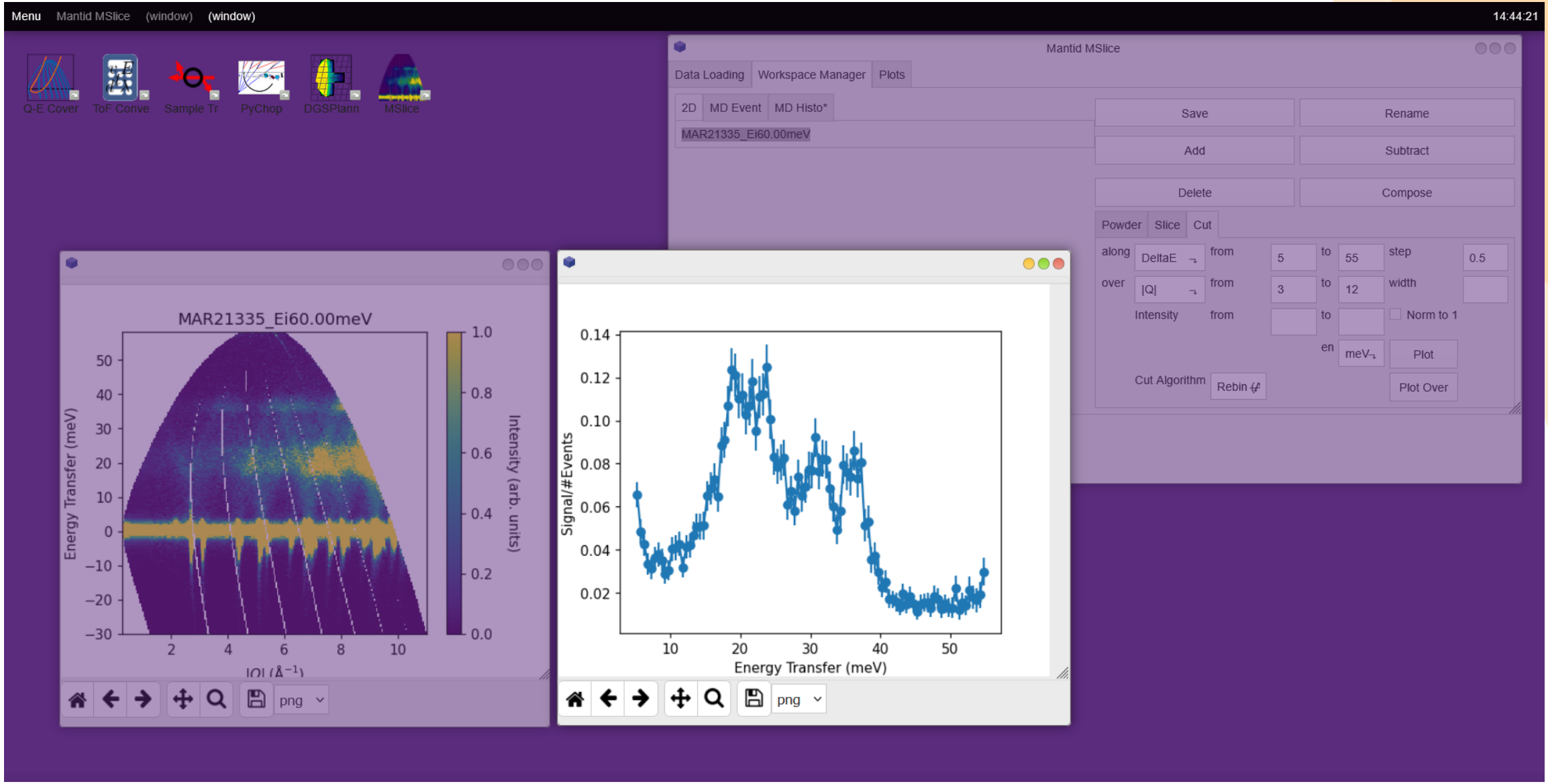
- Use `os.js` as the windowing manager (it is based on the `hyperapp` lightweight framework).
- Overrides `qtpy` namespace, mostly wrapping standard `html` widgets.

- Allows to port Python interfaces to run in the browser:
  - `ToFConverter`,
  - `QECoverage`,
  - `PyChop`,
  - `SampleTransmissionCalculator`,
  - `DGSPlanner`,
  - `MSlice`

- As such, cannot run C++ interfaces at the moment (need a shim for C++ Qt to html/js).


Menu
TofConverter
Sample Transmission Calculator
PyChopGUI
14:38:50
Q-E Cover
ToF Conve
Sample Tr
PyChop
DGSPlann
MSlice
TofConverter
Input value:
User specified
5000
Time of flight (microseconds)
Scattering Angle (Degrees):
90
Total flight path (Meters):
16
Convert To:
d-spacing (Angstroms)
Convert
Converted value:
0.8740292932628078
Sample Transmission Calculator
Type:
Single
Low:
0.1
Width:
0.1
High:
2
Chemical Formula
B
Mass Density
0.1
Thickness
0.1
Transmission (%)
Wavelength (Å)
png
?
Calculate
PyChopGUI
Instrument
MARI
Chopper
G
Frequency
400
Ei
180
Disk chopper slot index
0
Calculate and Plot
Hold current plot
Show multi-reps
Show data ascii window
Save data as ascii
Resolution
Flux-Ei
Flux-Freq
Time-Distance
Q-E
180.0
29.7
11.7
3.8
Distance (m)
TOF (μsec)
png
Reset original view
Number of frames to plot
1
Replot
First Rep Only


Menu (window)
14:41:29
Q-E Cover
ToF Conve
Samp
Instrument
ARCS
Incident Energy
10.0
Fast
Mask file
LoadMask
Goniometer
Name
Direction
Sense (+/-1)
Minim(deg)
Maxim(deg)
Step(deg)
Axis0 psi 0,1,0 1 0.0 90 15
Axis1 gl 0,0,1 1 0.0 0.0 1.0
Axis2 gs 1,0,0 1 0.0 0.0 1.0
Sample
gs
gl
psi
Beam
X
Z
png
Lattice parameters
a 4.2
b 4.3
c 5.6
alpha 90
beta 90
gamma 120
ux 1
uy 1
uz 0
vx 0
vy 0
vz 1
UB matrix
0.0000 -0.0000 0.1786
0.1358 -0.1358 -0.0000
0.2390 0.2316 0.0000
LoadIsawUB
LoadNexusUB
Min
Max
Step
Projection Basis
[H,0,0] [H,0,0] 0.050
[H,0,0] [H,0,0] 0.050
[H,0,0] [H,0,0]
[H,0,0] [H,0,0]
Projection u 1,0,0
Projection v 0,1,0
Projection w 0,0,1
Plot
Overplot
Color by angle
Aspect ratio 1:1
?
Save Figure
3.2
2.4
1.6
0.8
0.0
−0.8
−1.6
−2.4
−3.2
[0,K,0]
−3.0
−1.5
0.0
1.5
3.0
4.5
6.0
[H,0,0]
png


Menu
Mantid MSlice
(window)
(window)
14:44:21
Q-E Cover
ToF Conve
Sample Tr
PyChop
DGSPlann
MSlice
Mantid MSlice
Data Loading
Workspace Manager
Plots
2D
MD Event
MD Histo*
MAR21335_Ei60.00meV
Save
Rename
Add
Subtract
Delete
Compose
Powder
Slice
Cut
along
DeltaE
from
5
to
55
step
0.5
over
|Q|
from
3
to
12
width
Intensity
from
to
Norm to 1
en
meV
Plot
Cut Algorithm
Rebin
Plot Over
MAR21335_Ei60.00meV
Energy Transfer (meV)
Intensity (arb. units)
|Q| (Å⁻¹)
png
Signal/#Events
Energy Transfer (meV)
png

Future

- Add TBB library and re-enable threading in micromantid
- Make a jupyterlite version
- C++/wasm Qt wrapper

Questions?

Try it!